GLOWUP

An Online Cosmetics & Beauty Store

FINAL YEAR PROJECT DOCUMENTATION

(Software Requirements Specification, System Design,
Implementation & Testing)

Submitted By:

Hasnat Tariq

Roll No: **F22BDOCS1M01233**

Session: **2022 - 2026**

Submitted in Partial Fulfillment of the Requirements for the
Degree of Bachelor of Science in Computer Science

Supervisors Signature: _____

Version 1.0 — May 2026

Chapter 1: Introduction

1.1 Purpose

GlowUp is a full-stack web-based e-commerce platform developed as a final year project for the Bachelor of Science degree in Computer Science. The platform is designed to serve as an online cosmetics and beauty products store, catering primarily to female customers who are looking for skincare and makeup products suited to their individual skin types and beauty preferences. The system was built using the MERN stack — MongoDB, Express.js, React, and Node.js — and follows a RESTful API architecture.

This document serves as the official Software Requirements Specification (SRS) and project documentation for GlowUp. It describes the complete system from requirements gathering through design, implementation, and testing. The intended readers include the project supervisor, external examiners, and future developers who may maintain or extend the system.

GlowUp was conceived to solve a real-world problem: many women struggle to find beauty products that match their specific skin type. General-purpose online stores do not always make it easy to filter products by skin compatibility. GlowUp addresses this by building skin-type-aware product categorisation directly into the platform's core functionality, as visible in the product cards which display skin type tags such as DRY, SENSITIVE, OILY, COMBINATION, and NORMAL.

1.2 Scope

GlowUp is a standalone web application accessible from any modern browser. The system allows customers to browse and purchase beauty products across categories including Face Care, Eyes Care, Lips Care, Hair Care, Nails Care, Makeup Tools, New Arrivals, and All Products. The platform supports user registration, login, product browsing with search and skin-type filtering, a shopping cart with quantity controls and Stripe checkout, and order management.

The system consists of two major components: a React-based frontend (cosmetics-frontend) and a Node.js/Express backend (cosmetics-backend). The backend exposes a REST API that the frontend consumes. Product images are hosted on Cloudinary, and all application data is stored in a MongoDB database.

The following are explicitly outside the scope of this version: a native mobile application, an admin dashboard with analytics charts, a customer-facing product review submission interface, and any offline or physical store management functionality.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Meaning
MERN	MongoDB, Express.js, React, Node.js — the technology stack used to build the application
SRS	Software Requirements Specification — this document
REST / RESTful	Representational State Transfer — architectural style for designing web APIs
API	Application Programming Interface — the backend endpoints the frontend calls
JWT	JSON Web Token — a compact signed token used for stateless user authentication
MongoDB	A NoSQL document database; stores data as BSON (JSON-like) documents
Mongoose	An Object Data Modelling library for MongoDB and Node.js
Cloudinary	Cloud-based image hosting and management service used for product images
Stripe	Third-party online payment processing platform integrated for checkout
Redux / RTK	Redux Toolkit — state management library used in the React frontend
Vite	Fast frontend build tool used to bundle and serve the React application
bcryptjs	Node.js library used to hash user passwords before storage
Helmet	Express middleware that sets HTTP security headers
Rate Limiting	Security feature restricting API requests per client per time window
Morgan	HTTP request logger middleware for the Node.js backend
Multer	Node.js middleware for handling file uploads for product images

1.4 References

- React Documentation — https://react.dev
- Redux Toolkit Documentation — https://redux-toolkit.js.org
- Node.js Documentation — https://nodejs.org/en/docs
- Express.js Documentation — https://expressjs.com
- MongoDB Documentation — https://www.mongodb.com/docs

- Mongoose Documentation — https://mongoosejs.com/docs
- Cloudinary Documentation — https://cloudinary.com/documentation
- Stripe Checkout Documentation — https://stripe.com/docs/checkout
- Vite Documentation — https://vitejs.dev
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications

1.5 Overview

The remainder of this document is organised into three further chapters. Chapter 2 presents the complete Software Requirements Specification, covering all functional requirements, external interfaces, performance requirements, database requirements, design constraints, and quality attributes. Chapter 3 describes the system architecture and design, including architectural diagrams, module decomposition, data modelling, and a data dictionary derived from the actual Mongoose schemas. Chapter 4 covers implementation details and testing, listing all code modules, explaining database connectivity, presenting actual screenshots of the running application, and providing comprehensive test cases.

1.6 Product Perspective

GlowUp is an independent, self-contained web application. It does not form part of a larger existing system. In the broader market context it competes conceptually with platforms such as Sephora, Nykaa, and local Pakistani online beauty retailers. GlowUp differentiates itself by focusing specifically on skin-type-based product discovery — each product card visibly displays its compatible skin type tags (DRY, SENSITIVE, OILY, COMBINATION, NORMAL) so customers can immediately identify suitable products.

The system follows a classic three-tier architecture. The presentation tier is a React single-page application built with Vite and styled using Tailwind CSS. The application tier is a Node.js server running Express.js. The data tier is a MongoDB Atlas cloud database accessed via Mongoose. Product media assets are managed through Cloudinary.

1.6.1 System Interfaces

GlowUp interacts with the following external systems:

- MongoDB Atlas — cloud-hosted NoSQL database for all persistent application data
- Cloudinary — cloud image storage; images uploaded via Multer and the Cloudinary Node.js SDK
- Stripe — third-party payment gateway; backend creates Stripe Checkout Sessions with 'Checkout with Stripe' redirect flow

1.6.2 User Interfaces

The system provides a graphical web interface built with React 19 and Tailwind CSS 4. The interface uses a soft pink and rose colour palette appropriate for a cosmetics brand. Navigation is handled by React Router DOM v7. The category navigation bar displays: Face Care, Eyes Care, Lips Care, Hair Care, Nails Care, Makeup Tools, New Arrivals, and All Products.

1.6.3 Hardware Interfaces

The application has no direct hardware interface requirements. End users access the system through standard computing devices with an internet-connected browser. The backend server requires a hosting environment capable of running Node.js v20 or later.

1.6.4 Software Interfaces

Component	Version	Role in System
React	19.2.5	Frontend UI library; component-based rendering
React Router DOM	7.14.2	Client-side routing between pages
Redux Toolkit	2.11.2	Global state management (products, auth, cart)
Axios	1.15.2	HTTP client in api.js to call backend endpoints
React Toastify	11.1.0	Toast notification library for user feedback
Tailwind CSS	4.2.4	Utility-first CSS framework for styling
Vite	8.0.9	Frontend build tool and development server
Node.js	20+	Backend JavaScript runtime
Express.js	4.18.2	Web framework for building the REST API
Mongoose	8.0.0	ODM for MongoDB; schema definition and queries
bcryptjs	2.4.3	Password hashing with 10 salt rounds
jsonwebtoken	9.0.0	JWT generation and verification for auth
Cloudinary SDK	1.37.3	Image upload to Cloudinary cloud storage
Multer	1.4.5-lts.1	Multipart form data handling for image uploads
Stripe	12.0.0	Payment via Stripe Checkout Sessions
Helmet	7.0.0	HTTP security headers middleware
express-rate-limit	6.7.0	Rate limiting — 100 requests per 15-minute window
Morgan	1.10.0	HTTP request logging in

		development
dotenv	16.0.3	Environment variable loading from .env file

1.6.5 Communications Interfaces

The React frontend communicates with the Node.js backend over HTTP using Axios. All requests and responses use JSON format. The base URL is configured in src/services/api.js. The api.js interceptor automatically attaches the JWT Bearer token from localStorage to every outgoing request header.

1.6.6 Memory Constraints

The backend server requires a minimum of 512 MB RAM. The MongoDB Atlas free tier provides 512 MB storage, sufficient for the current product catalogue. Image storage is offloaded to Cloudinary so database storage is not impacted by media assets.

1.6.7 Operations

The platform operates 24 hours a day. The backend is started via npm start in production or npm run dev during development. The rate limiter allows a maximum of 100 requests per 15-minute window per IP address.

1.6.8 Site Adaptation Requirements

Before deployment the following environment variables must be configured in the backend .env file:

- MONGO_URI — MongoDB Atlas connection string
- JWT_SECRET — secret key for signing and verifying JWT tokens
- CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET — Cloudinary credentials
- STRIPE_SECRET_KEY — Stripe secret key for payment processing
- PORT — backend server port (default 5000)

1.7 Product Functions

- User Registration — new customers create an account with name, email, and password
- User Login and Authentication — registered users log in; JWT token issued and stored in localStorage
- Product Browsing — products fetched from backend and displayed in a responsive grid
- Search and Skin-Type / Category Filtering — customers search by name and filter by category or skin type
- Product Detail View — dedicated page showing full product info, skin type tags, quantity selector, and Add to Cart button
- Shopping Cart — customers add products, update quantities with +/- controls, remove items; subtotal calculated automatically
- Checkout with Stripe — Stripe Checkout Session created; user redirected to Stripe's hosted page

- Order Management — orders saved in database with items, shipping address, total price, and payment info
- Admin Controls — admin users can create, update, delete products and manage order statuses

1.8 User Characteristics

GlowUp targets two user types. The primary user is a female customer aged 18–40 who shops online for cosmetics. This user is comfortable with standard e-commerce flows and does not require technical knowledge. The secondary user is an administrator who manages the product catalogue and orders through the backend API.

1.9 Constraints

- Application must run on Node.js version 20 or later on the backend
- Frontend must be built using React and Tailwind CSS
- All passwords must be hashed using bcryptjs with minimum 10 salt rounds; plain text storage is strictly prohibited
- All protected API endpoints must validate the JWT token via authMiddleware
- System must not store Stripe card details; all payment data handled by Stripe
- Product images must be uploaded to Cloudinary; only image URLs stored in MongoDB
- Backend must apply Helmet security headers and enforce rate limiting on all routes

1.10 Assumptions and Dependencies

- MongoDB Atlas cluster is available and accessible at the configured MONGO_URI
- Valid Cloudinary account credentials are configured in environment variables
- Stripe account is active and STRIPE_SECRET_KEY is valid
- End users have a stable internet connection and use a modern browser
- Frontend depends on backend API being available at the configured base URL

1.11 Apportioning of Requirements

The following features were deferred to future versions due to time constraints:

- Version 2.0: Admin dashboard with sales analytics and inventory management UI
- Version 2.0: Customer product ratings and reviews on product detail page
- Version 2.0: Wishlist / saved items functionality
- Version 3.0: Native mobile application for Android and iOS
- Version 3.0: AI-based personalised skincare recommendation engine

Chapter 2: Software Requirements Specification

This chapter presents the complete set of functional and non-functional requirements for the GlowUp system. Every requirement is assigned a unique identifier. Requirements marked with 'shall' are mandatory.

2.1 Specific Requirements

2.1.1 User Registration (REQ-AUTH-01 to REQ-AUTH-04)

- REQ-AUTH-01: The system shall allow a new user to register by submitting name, email, and password via POST /api/auth/register.
- REQ-AUTH-02: The system shall validate that all three fields are present; if any is missing, return HTTP 400: 'All fields are required'.
- REQ-AUTH-03: The system shall reject duplicate emails with HTTP 400: 'User already exists'.
- REQ-AUTH-04: The system shall hash passwords using bcryptjs with 10 salt rounds via a Mongoose pre-save hook before persisting to MongoDB.

2.1.2 User Login and Authentication (REQ-AUTH-05 to REQ-AUTH-08)

- REQ-AUTH-05: The system shall allow registered users to log in via POST /api/auth/login with email and password.
- REQ-AUTH-06: The system shall use the matchPassword instance method on the User model (bcrypt.compare) to verify the password against the stored hash.
- REQ-AUTH-07: On successful login, the system shall sign and return a JWT token with the user's ID and role, expiring in 7 days.
- REQ-AUTH-08: The protect middleware shall extract the Bearer token from the Authorization header, verify it using JWT_SECRET, and attach the authenticated user to req.user.

2.1.3 Role-Based Access Control (REQ-AUTH-09 to REQ-AUTH-10)

- REQ-AUTH-09: The User model shall support two roles: 'user' (default) and 'admin', enforced as an enum at schema level.
- REQ-AUTH-10: The admin middleware shall return HTTP 403 'Admin access denied' if the authenticated user does not have the admin role.

2.1.4 Product Browsing, Search, and Skin-Type Filtering (REQ-PROD-01 to REQ-PROD-05)

- REQ-PROD-01: The system shall expose GET /api/products returning a paginated product list. Optional query params: search, category, page (default 1), limit (default 10).
- REQ-PROD-02: When search is provided, filter by name using a case-insensitive MongoDB regex ($regex, $options: 'i').
- REQ-PROD-03: When category is provided, filter by exact category match. Categories include Face Care, Eyes Care, Lips Care, Hair Care, Nails Care, Makeup Tools.
- REQ-PROD-04: The response shall include products array, total count, current page, and total pages.
- REQ-PROD-05: GET /api/products/:id shall return a single product; return HTTP 404 if not found.

2.1.5 Skin Type Tags on Products (REQ-PROD-06)

- REQ-PROD-06: Each product shall store a skinType array containing applicable skin types (e.g., DRY, SENSITIVE, OILY, COMBINATION, NORMAL). These tags are displayed on product cards and the product detail page to help customers identify suitable products.

2.1.6 Product Management — Admin (REQ-PROD-07 to REQ-PROD-10)

- REQ-PROD-07: POST /api/products (admin only) shall create a new product. Images uploaded via Multer shall be stored on Cloudinary; their URLs saved in the product's images array.
- REQ-PROD-08: PUT /api/products/:id (admin only) shall update product fields.
- REQ-PROD-09: DELETE /api/products/:id (admin only) shall delete the product and destroy all associated Cloudinary images.
- REQ-PROD-10: The Product schema shall support a ratings sub-array for a future review system.

2.1.7 Shopping Cart (REQ-CART-01 to REQ-CART-03)

- REQ-CART-01: The frontend shall maintain cart state using a Redux cartSlice. The cart displays a 'Your Bag' panel showing product image, name, price, and quantity controls.
- REQ-CART-02: Each cart item shall store product reference and quantity. The subtotal shall be calculated and displayed (e.g., $35.00 for 1 × Matte Bronzing Powder).
- REQ-CART-03: Users shall add items via the Add to Cart button on the product detail page, update quantities with +/- buttons, and remove items with the X button.

2.1.8 Checkout and Payment (REQ-PAY-01 to REQ-PAY-04)

- REQ-PAY-01: POST /api/payment/checkout (protected) shall accept cart items and create a Stripe Checkout Session.
- REQ-PAY-02: The Stripe session shall use payment_method_types: ['card'], mode: 'payment', and line items derived from cart data.
- REQ-PAY-03: The API shall return the Stripe session URL; the frontend shall display a 'Checkout with Stripe' button that redirects to this URL.
- REQ-PAY-04: Success and cancel redirect URLs shall default to localhost:3000/success and localhost:3000/cancel.

2.1.9 Order Management (REQ-ORD-01 to REQ-ORD-05)

- REQ-ORD-01: POST /api/orders (protected) shall validate items, check stock, calculate total, decrement stock, and create the order.
- REQ-ORD-02: If stock is insufficient, the system shall return HTTP 400: 'Insufficient stock'.
- REQ-ORD-03: On success, product stock fields shall be decremented by ordered quantities.
- REQ-ORD-04: GET /api/orders/user (protected) shall return all orders for the authenticated user with product details populated.
- REQ-ORD-05: Admin endpoints shall allow viewing all orders and updating order status (pending, processing, shipped, delivered, cancelled).

2.2 External Interfaces

Interface	Type	Description / Format
POST /api/auth/register	API Input	Body: { name, email, password }
POST /api/auth/login	API Input	Body: { email, password }; Returns: { token, user }
GET /api/products	API Output	Query: search, category, page, limit; Returns: { products[], total, page, pages }

GET /api/products/:id	API Output	Returns single product document or 404
POST /api/products	API Input	Multipart: name, description, price, category, skinType[], stock, images[]
POST /api/payment/checkout	API Input	Body: { items[], success_url, cancel_url }; Returns: { id, url }
POST /api/orders	API Input	Body: { items[], shippingAddress, paymentInfo }; Returns: order document
GET /api/orders/user	API Output	Returns array of user orders with populated product details
Authorization Header	Security	All protected routes require: 'Authorization: Bearer <token>'
Cloudinary Upload	External	cloudinary.uploader.upload() returns secure_url and public_id
Stripe Checkout	External	stripe.checkout.sessions.create() returns session.id and session.url

2.3 Functions

- Authentication and Session Management: authController handles registration and login. Passwords hashed via pre-save hook. JWT issued on login and verified on every protected request.
- Product Catalogue Management: productController handles CRUD. Search and category filtering use dynamic MongoDB query objects. Image management via Cloudinary.
- Cart State Management: Redux cartSlice manages cart on the frontend. Add, update, and remove items. Totals calculated locally.
- Payment Processing: paymentController creates Stripe Checkout Session; returns redirect URL. All sensitive payment handling stays on Stripe.
- Order Processing: orderController validates items, checks stock, calculates totals, decrements stock, persists order.
- Error Handling: errorMiddleware provides global Express error handler via next(err).

2.4 Performance Requirements

- PERF-01: 90% of API responses for product listing and search shall complete within 1.5 seconds under normal load.
- PERF-02: System shall support at least 50 concurrent users; rate limiter set to 100 requests per 15-minute window.
- PERF-03: Stripe Checkout Session creation shall complete within 3 seconds.
- PERF-04: Frontend page transitions shall complete within 2 seconds on broadband, using Vite's optimised production build.

2.5 Logical Database Requirements

- users — customer and admin account data
- products — complete product catalogue including skin type tags, images, and ratings
- orders — customer orders with items, shipping address, total price, and payment information

2.6 Design Constraints

- Backend must use Node.js with Express.js
- Database must be MongoDB accessed via Mongoose
- Frontend must be a React SPA built with Vite
- All API communication must use JSON
- Product images must be stored on Cloudinary; no local file storage in production
- Environment variables used for all sensitive configuration

2.6.1 Standards Compliance

- All backend API endpoints follow REST conventions
- HTTP status codes follow standard semantics: 200, 201, 400, 401, 403, 404, 500
- Passwords stored using bcrypt per standard credential storage practices
- Helmet middleware enforces HTTP security header standards

2.7 Software System Attributes

2.7.1 Reliability

All controller functions wrap logic in try-catch blocks forwarding errors to errorMiddleware via next(err). The db.js connectDB function calls process.exit(1) on connection failure.

2.7.2 Availability

The stateless Node.js backend can be restarted without data loss. Designed for deployment on cloud platforms supporting automatic restarts.

2.7.3 Security

- Passwords hashed with bcryptjs using 10 salt rounds
- JWT tokens verified on all protected routes
- HTTP security headers applied by Helmet
- Rate limiting: 100 requests per 15 minutes per IP
- Admin routes protected by both protect and admin middleware
- Sensitive credentials stored in .env, excluded from version control

2.7.4 Maintainability

Clear separation of concerns: config/, controllers/, middleware/, models/, routes/ on backend; components/, pages/, redux/, routes/, services/ on frontend.

2.7.5 Portability

Frontend deployable to any static host (Vercel, Netlify) after vite build. Backend deployable to any Node.js platform (Render, Railway). Environment variables ensure portability across environments.

Chapter 3: System Design and Architecture

3.1 Architectural Design

GlowUp follows a three-tier client-server architecture:

- Presentation Tier: React SPA in the user's browser. Renders UI, manages state via Redux, communicates with backend via Axios.
- Application Tier: Node.js/Express server. Handles business logic, authentication, REST API, and external service integration.
- Data Tier: MongoDB Atlas. Stores all persistent data including users, products, and orders.

Tier	Technology	Key Responsibilities
Presentation	React 19 + Tailwind CSS 4 + Vite	UI rendering, routing (React Router), state (Redux), API calls (Axios)
Application	Node.js 20 + Express.js 4	REST API, authentication (JWT), business logic, external service integration
Data	MongoDB Atlas + Mongoose 8	Data persistence, schema validation, document relationships
Media Storage	Cloudinary	Product image hosting and delivery
Payment	Stripe	Secure payment via hosted Checkout Sessions

3.2 Decomposition Description

3.2.1 Backend Module Structure

File / Directory	Purpose
server.js	Entry point. Loads env vars, initialises Express, registers all middleware (cors, helmet, morgan, rate limiter, express.json), connects to MongoDB, mounts all route handlers.
config/db.js	Exports connectDB() which calls mongoose.connect() using MONGO_URI. Calls process.exit(1) on failure.
config/cloudinary.js	Configures Cloudinary SDK using environment credentials. Exports the configured instance.
models/User.js	User schema: name, email, password, role, timestamps.

	Pre-save hook hashes password. matchPassword instance method for login.
models/Product.js	Product schema: name, description, price, category, stock, images[{url,public_id}], ratings[{user,rating,comment}], timestamps.
models/Order.js	Order schema: user ref, items[{product ref, quantity}], totalPrice, status enum, shippingAddress, paymentInfo, timestamps.
controllers/authController.js	Exports register (validate, check duplicate, create user, sign JWT) and login (find user, verify password, sign JWT).
controllers/productController.js	Exports getProducts (search/category/pagination), getProductById, createProduct (Cloudinary upload), updateProduct, deleteProduct (Cloudinary cleanup).
controllers/orderController.js	Exports createOrder (stock check, total calc, stock decrement), getUserOrders, getAllOrders (admin), updateOrderStatus (admin).
controllers/paymentController.js	Exports createCheckoutSession: builds Stripe line_items, calls stripe.checkout.sessions.create(), returns session id and url.
middleware/authMiddleware.js	Exports protect (JWT verify, attach req.user) and admin (check role === 'admin').
middleware/errorMiddleware.js	Global Express error handler; catches errors from next(err) and returns JSON response.
routes/authRoutes.js	POST /register → register; POST /login → login.
routes/productRoutes.js	GET /, GET /:id, POST / (admin+multer), PUT /:id (admin+multer), DELETE /:id (admin).
routes/orderRoutes.js	POST / (protect), GET /user (protect), GET / (admin), PUT /:id/status (admin).
routes/paymentRoutes.js	POST /checkout (protect) → createCheckoutSession.

3.2.2 Frontend Module Structure

File / Directory	Purpose
src/main.jsx	Entry point. Wraps app in Redux Provider and BrowserRouter, mounts App.
src/App.jsx	Root component: Navbar + AppRoutes + Footer.
src/routes/AppRoutes.jsx	Defines all routes: /, /login, /register, /products, /product/:id, /cart, /checkout.
src/services/api.js	Axios instance (baseURL: localhost:5000/api). Interceptor attaches JWT Bearer token from localStorage.

src/redux/store.js	Redux store combining auth, product, and cart reducers.
src/redux/Productslice.js	productSlice with fetchProducts async thunk calling GET /api/products.
src/pages/Home.jsx	Homepage: hero section 'Reveal Your Inner Radiance', Spring Collection badge, Shop the Collection button, product grid.
src/pages/Products.jsx	Product listing: search input + category dropdown + product grid with skin type tags on each card.
src/pages/ProductDetail.jsx	Single product: large image, name, price (e.g. $35.00), skin type tags (e.g. Combination, Normal), description, quantity selector, Add to Cart button.
src/pages/Cart.jsx	'Your Bag' panel: product image, name, price, quantity +/- controls, remove X, subtotal, 'Checkout with Stripe' button.
src/pages/Checkout.jsx	Checkout form: collects shipping address; calls POST /api/payment/checkout; redirects to Stripe.
src/pages/Login.jsx	'Welcome Back' login form: Email Address field, Password field, Sign In button, Forgot Password link, Sign Up link.
src/pages/Register.jsx	Registration form: name, email, password fields; Register button.
src/components/Navbar.jsx	Navigation bar with GLOW UP brand, category nav (Face Care, Eyes Care, etc.), search, account, cart icons.
src/components/ProductCard.jsx	Product card: image with category badge, product name, description, skin type tags, price, Add button.
src/components/Footer.jsx	Footer with copyright notice.
src/components/Loader.jsx	Loading spinner shown during API calls.
src/components/Notifications.jsx	React Toastify wrapper for user feedback notifications.

3.3 Design Rationale

React was chosen for its component-based model which maps naturally to e-commerce UI elements. Vite was selected over Create React App for its significantly faster development server. Tailwind CSS enables rapid consistent styling without custom CSS files.

Redux Toolkit was chosen over Context API because multiple pieces of shared state (auth, products, cart) are accessed and updated across many components. RTK's createSlice and createAsyncThunk reduce boilerplate significantly.

Node.js with Express allows the entire project to be written in JavaScript. MongoDB was chosen because the product catalogue has variable attributes across categories. Stripe was integrated for payment to avoid PCI compliance complexity. Cloudinary offloads image storage from the server.

3.4 Data Description

All data stored in MongoDB as BSON documents in three collections: users, products, orders. Mongoose schemas enforce structure and data types. Inter-collection relationships maintained using ObjectId references. Mongoose populate() resolves references when querying orders.

3.5 Data Dictionary

Field	Collection	Type	Required	Description
_id	All	ObjectId	Auto	MongoDB auto-generated unique document identifier
createdAt	All	Date	Auto	Timestamp set automatically by Mongoose timestamps option
updatedAt	All	Date	Auto	Timestamp updated automatically on every save
name	users	String	Yes	Full name of the registered user
email	users	String	Yes	Unique email address; used as login identifier
password	users	String	Yes	bcrypt hash of password (never stored in plain text)
role	users	String	No	Enum: 'user' (default) or 'admin'
name	products	String	Yes	Display name (e.g., 'Matte Bronzing Powder')
description	products	String	Yes	Full product description
price	products	Number	Yes	Price in USD (e.g., 35.00 for Matte Bronzing Powder)
category	products	String	Yes	Category: Face Care, Eyes Care, Lips Care, Hair

				Care, Nails Care, Makeup Tools
skinType	products	Array	No	Array of compatible skin types: DRY, SENSITIVE, OILY, COMBINATION, NORMAL
stock	products	Number	Yes	Units available; decremented on each order
images	products	Array	No	Array of { url: String, public_id: String } from Cloudinary
ratings	products	Array	No	Array of { user: ObjectId, rating: Number, comment: String }
user	orders	ObjectId	Yes	Reference to users collection — who placed the order
items	orders	Array	Yes	Array of { product: ObjectId (ref products), quantity: Number }
totalPrice	orders	Number	Yes	Total order value, calculated server-side
status	orders	String	No	Enum: pending (default), processing, shipped, delivered, cancelled
shippingAddress	orders	Object	No	{ address, city, postalCode, country }
paymentInfo	orders	Object	No	{ id: Stripe session ID, status: payment status string }

3.6 Component Design

authController — register

1. Receive name, email, password from req.body
2. If any field missing → return 400: 'All fields are required'
3. Query User.findOne({ email }) for duplicate check

4. If duplicate → return 400: 'User already exists'
5. Call User.create() — pre-save hook hashes password automatically
6. Sign JWT: jwt.sign({ id, role }, JWT_SECRET, { expiresIn: '7d' })
7. Return 201 with { token, user: { id, name, email, role } }

productController — getProducts

8. Read query params: search, category, page (default 1), limit (default 10)
9. Build query object: if search → { name: { $regex: search, $options: 'i' } }; if category → add { category }
10. Call Product.find(query).skip((page-1)*limit).limit(Number(limit))
11. Call Product.countDocuments(query) for pagination
12. Return { products, total, page, pages: Math.ceil(total/limit) }

orderController — createOrder

13. Check items array is non-empty → else 400: 'No order items'
14. For each item: find product, verify stock >= quantity, accumulate totalPrice, decrement stock, save
15. Create Order: { user: req.user._id, items, totalPrice, shippingAddress, paymentInfo }
16. Return 201 with created order

paymentController — createCheckoutSession

17. Map items to Stripe line_items: { price_data: { currency, product_data: { name }, unit_amount: price*100 }, quantity }
18. Call stripe.checkout.sessions.create({ payment_method_types, line_items, mode: 'payment', success_url, cancel_url })
19. Return { id: session.id, url: session.url }

Chapter 4: Implementation and Testing

4.1 Code Modules

Module	Location	Lang	Description
server.js	cosmetics-backend/	JS	Express setup, middleware, route mounting, server start
config/db.js	cosmetics-backend/ config/	JS	MongoDB Atlas connection via Mongoose
config/cloudinary.js	cosmetics-backend/ config/	JS	Cloudinary SDK configuration
models/User.js	cosmetics-backend/ models/	JS	User schema with bcrypt pre-save hook and matchPassword method
models/Product.js	cosmetics-backend/ models/	JS	Product schema with images array, skinType array, ratings sub-array

models/Order.js	cosmetics-backend/models/	JS	Order schema with status enum and shipping/payment sub-objects
controllers/authController.js	cosmetics-backend/controllers/	JS	Register and login logic with JWT signing
controllers/productController.js	cosmetics-backend/controllers/	JS	Full CRUD for products with Cloudinary integration
controllers/orderController.js	cosmetics-backend/controllers/	JS	Order creation with stock validation and admin views
controllers/paymentController.js	cosmetics-backend/controllers/	JS	Stripe Checkout Session creation
middleware/authMiddleware.js	cosmetics-backend/middleware/	JS	JWT protect middleware and admin role check
middleware/errorMiddleware.js	cosmetics-backend/middleware/	JS	Global Express error handler
routes/authRoutes.js	cosmetics-backend/routes/	JS	POST /register and POST /login endpoints
routes/productRoutes.js	cosmetics-backend/routes/	JS	Full product CRUD routes with Multer file upload
routes/orderRoutes.js	cosmetics-backend/routes/	JS	Order creation, retrieval, and status update routes
routes/paymentRoutes.js	cosmetics-backend/routes/	JS	Stripe checkout session route
src/main.jsx	cosmetics-frontend/src/	JSX	React entry point with Redux Provider and BrowserRouter
src/App.jsx	cosmetics-frontend/src/	JSX	Root layout: Navbar + AppRoutes + Footer
src/routes/AppRoutes.jsx	cosmetics-frontend/src/routes/	JSX	React Router route definitions for all pages
src/services/api.js	cosmetics-frontend/src/services/	JS	Axios instance with JWT interceptor
src/redux/store.js	cosmetics-frontend/src/redux/	JS	Redux store combining auth, product, cart slices
src/redux/Productslice.js	cosmetics-frontend/src/redux/	JS	Async thunk for fetching products; product state
src/pages/Home.jsx	cosmetics-frontend/src/pages/	JSX	Homepage with hero and featured product grid

src/pages/Products.jsx	cosmetics-frontend/src/pages/	JSX	Product listing with search, category filter, skin type tags
src/pages/ProductDetail.jsx	cosmetics-frontend/src/pages/	JSX	Single product view with skin type tags and Add to Cart
src/pages/Cart.jsx	cosmetics-frontend/src/pages/	JSX	'Your Bag' panel with quantity controls and Stripe checkout button
src/pages/Checkout.jsx	cosmetics-frontend/src/pages/	JSX	Checkout form and Stripe redirect
src/pages/Login.jsx	cosmetics-frontend/src/pages/	JSX	'Welcome Back' login form
src/pages/Register.jsx	cosmetics-frontend/src/pages/	JSX	Registration form
src/components/Navbar.jsx	cosmetics-frontend/src/components/	JSX	Navigation bar with category links and icons
src/components/ProductCard.jsx	cosmetics-frontend/src/components/	JSX	Product card with image, skin type tags, price, Add button
src/components/Footer.jsx	cosmetics-frontend/src/components/	JSX	Footer with copyright
src/components/Loader.jsx	cosmetics-frontend/src/components/	JSX	Loading spinner during API calls
src/components/Notifications.jsx	cosmetics-frontend/src/components/	JSX	React Toastify wrapper

4.2 Database Connectivity

The backend connects to MongoDB Atlas via Mongoose in config/db.js. The connection is established at server startup using the MONGO_URI environment variable. All controllers share the same connection through Mongoose's connection pooling. Mongoose models (User, Product, Order) translate JavaScript method calls into MongoDB driver operations automatically.

A seed route at GET /api/seed-products is available in server.js to populate the database with five sample products (Velvet Matte Lipstick, Hydrating Foundation, Glow Face Cream, Waterproof Mascara, Rose Blush Compact) for testing. The actual product catalogue shown in the application screenshots (Champagne Glow Highlighter, Matte Bronzing Powder, Sunset Glow Eyeshadow Palette, etc.) was added through the admin product creation endpoint.

4.3 Overview of User Interface

The GlowUp user interface uses a soft pink and rose colour theme consistent with a premium cosmetics brand. A persistent navigation bar shows the GLOW UP brand in a serif font, social media icons on the left, and search, account, and cart icons on the right. A secondary navigation bar in warm rose displays the product categories: Face Care, Eyes Care, Lips Care, Hair Care, Nails Care, Makeup Tools, New Arrivals, and All Products.

The homepage features a prominent hero section with 'The Spring Collection' badge, the headline 'Reveal Your Inner Radiance' in a bold serif font, a descriptive subheading, and a 'Shop the Collection' call-to-action button. The All Products page displays a responsive three-column grid of product cards, each showing a high-quality product image with a category badge, product name, short description, skin type compatibility tags, price, and an Add to Cart button. Clicking a product navigates to the Product Detail page which shows the full-size image, name, price, skin type tags, description, a quantity selector, and a prominent 'Add to Cart' button showing the product price. The Your Bag panel shows cart items with thumbnail images, names, prices, quantity +/- controls, a remove button, subtotal, and a 'Checkout with Stripe' button. The Login page presents a clean centred card with 'Welcome Back' heading, email and password fields, a Sign In button, a Forgot Password link, and a Sign Up link for new users.

4.4 Test Cases

ID	Module	Test Description	Input	Expected Result	Status
TC-01	Registration	Successful new user registration	Valid name, email, unused password	HTTP 201; JWT token returned; user saved with hashed password	Pass
TC-02	Registration	Duplicate email rejected	Already registered email	HTTP 400: 'User already exists'	Pass
TC-03	Registration	Missing fields validation	Email and password empty	HTTP 400: 'All fields are required'	Pass
TC-04	Registration	Password hashed in database	Register with any password	users collection stores bcrypt hash, never plain text	Pass
TC-05	Login	Correct credentials accepted	Registered email + matching password	HTTP 200; JWT with 7-day expiry returned	Pass

TC-06	Login	Wrong password rejected	Correct email, wrong password	HTTP 400: 'Invalid credentials'	Pass
TC-07	Login	Non-existent email rejected	Email not in database	HTTP 400: 'Invalid credentials'	Pass
TC-08	Auth Middleware	Protected route blocked without token	GET /api/orders/user with no Authorization header	HTTP 401: 'Not authorized, no token'	Pass
TC-09	Auth Middleware	Protected route accessible with valid token	Valid Bearer token	HTTP 200; correct data returned	Pass
TC-10	Auth Middleware	Admin route blocked for regular user	DELETE /api/products/:id with user token	HTTP 403: 'Admin access denied'	Pass
TC-11	Products	Fetch all products	GET /api/products	HTTP 200; products[], total, page, pages returned	Pass
TC-12	Products	Search by name	GET /api/products?search=bronzing	Only 'Matte Bronzing Powder' returned (case-insensitive)	Pass
TC-13	Products	Filter by category	GET /api/products?category=Face Care	Only Face Care products returned	Pass
TC-14	Products	Search with no results	GET /api/products?search=zzzznotexist	HTTP 200; products array empty; total: 0	Pass
TC-15	Products	Get single product	GET /api/products/:validId	HTTP 200; Matte Bronzing Powder document returned	Pass
TC-16	Products	Invalid product ID	GET /api/products/invalidId	HTTP 404: 'Product not found'	Pass
TC-17	Products	Create product (admin)	POST /api/products with admin token + valid body	HTTP 201; product saved; images on Cloudinary	Pass

TC-18	Products	Delete product (admin)	DELETE /api/products/:id with admin token	HTTP 200: 'Product deleted successfully'; Cloudinary images removed	Pass
TC-19	Orders	Place valid order	POST /api/orders with valid items + address	HTTP 201; order created; product stock decremented	Pass
TC-20	Orders	Insufficient stock	Quantity exceeding product.stock	HTTP 400: 'Insufficient stock'	Pass
TC-21	Orders	Empty order rejected	items: []	HTTP 400: 'No order items'	Pass
TC-22	Orders	Get user orders	GET /api/orders/user with user token	HTTP 200; user's orders with product details populated	Pass
TC-23	Orders	Admin update status	PUT /api/orders/:id/status { status: 'shipped' }	HTTP 200; order status updated to 'shipped'	Pass
TC-24	Payment	Create Stripe session	POST /api/payment/checkout with cart items	HTTP 200; { id: stripeSessionId, url: stripeCheckoutUrl }	Pass
TC-25	Payment	Empty cart checkout rejected	items: []	HTTP 400: 'No items to checkout'	Pass
TC-26	Cart UI	Add product to cart	Click 'Add to Cart • $35.00' on Matte Bronzing Powder	Product appears in Your Bag; subtotal shows $35.00; toast shown	Pass
TC-27	Cart UI	Remove item from cart	Click X on cart item	Item removed from Your Bag; subtotal resets to $0.00	Pass
TC-28	Cart UI	Update quantity	Click + on cart item	Quantity increments; subtotal updates (e.g. $70.00 for qty 2)	Pass
TC-29	Cart UI	Checkout with Stripe button	Click 'Checkout with Stripe'	POST /api/payment/ch	Pass

				eckout called; browser redirected to Stripe	
TC-30	Login UI	Sign In with correct credentials	Valid email + password, click Sign In	JWT stored in localStorage; redirected to Homepage	Pass

— End of GlowUp Final Year Project Documentation —

Version 1.0 | May 2026 | Bachelor of Science in Computer Science

Hasnat Tariq | Roll No: F22BDOCS1M01233